

B1-2 : 평가문항 대비자료

평가 항목 1~4 — 실행한 내용 & 설명 Q&A

항목1: 체크리스트 충족 / 항목2: 사용 명령어·도구 / 항목3: 원리 설명 / 항목4: 운영 적용·개선 제안

항목 1 — 평가 체크리스트 충족 현황 (8/8)

3건의 GitHub Issue(report-case1~3)에 아래 8개 항목이 모두 근거와 함께 포함되어 있음

#	평가 항목	충족 근거
1	[OOM] 메모리 선형 증가 → 강제종료 로그	agent_app.log: +25MB 반복 → MemoryGuard 발동
2	[OOM] MEMORY_LIMIT 조정 Before/After	256MB → 512MB, 생존 시간 연장 확인
3	[CPU] CPU 임계치 초과 → 종료 로그	Watchdog: 99.7% > 50% → SIGTERM
4	[CPU] CPU_MAX_OCCUPY 조정 Before/After	50% → 80~90%, 종료 조건 변화 확인
5	[Deadlock] PID 생존 + 로그 멈춤 식별	ps -ef PID 28648, TIME 불변, 로그 정지
6	[Deadlock] MULTI_THREAD_ENABLE Before/After	true → false, 재현/회피 비교
7	[Format] GitHub Issue 구조 3건	현상→증거→원인→조치 5단 구조 동일 적용
8	[Evidence] PID·타임스탬프·로그 근거 첨부	각 리포트 'Evidence & Logs' 섹션에 포함

항목 2-① — monitor.sh 메모리 증가 패턴 추적 명령어

Q. monitor.sh에서 메모리 증가 패턴을 추적하기 위해 사용한 명령어와 데이터 추출 방법은?

```
# monitor.sh — 메모리 사용률(%) 계산
MEM_TOTAL=$(free | grep Mem | awk '{print $2}') # 전체 메모리(KB)
MEM_USED=$(free | grep Mem | awk '{print $3}') # 사용 중 메모리(KB)
MEM_USAGE=$(awk "BEGIN{printf \"%.1f\", ($MEM_USED/$MEM_TOTAL)*100}")

# 1줄씩 누적 기록 (매분 cron 실행)
echo "[$TIMESTAMP] PID:$PID CPU:$CPU_USAGE% MEM:${MEM_USAGE}% ..." >> monitor.log
```

free 명령으로 전체/사용 메모리(KB)를 읽고, awk로 사용률(%) = (사용량/전체)×100 을 계산

awk를 쓰는 이유: 셸(bash)은 정수 연산만 가능 → 소수점(.1f) 계산은 awk에 위임

cron으로 매분 자동 실행 → monitor.log에 한 줄씩 누적(>>) → 시계열 데이터 형성

데이터 추출: grep "MEM:" monitor.log 로 MEM% 값만 모아 시간순으로 비교 → '증가 패턴' 확인

항목 2-② — CPU 사용률 확인 도구 & 옵션의 의미

Q. 프로세스의 CPU 사용률을 확인하기 위해 선택한 도구와 적용한 옵션의 의미는?

도구 / 명령	옵션	의미
top -bn1	-b (batch mode)	대화형 화면 없이 결과를 텍스트로 출력 → 스크립트에서 파싱 가능
top -bn1	-n1 (iterations=1)	1회만 측정 후 즉시 종료 (기본은 무한 반복)
ps -eo pid,comm,%cpu,%mem	-e -o	모든 프로세스(e)를 지정한 컬럼(o) 형식으로 출력
agent-leak-sim.py: os.times()	user+sys 시간	프로세스 자신의 CPU 사용 시간 / 경과시간(wall) → 실제 점유율(%) 직접 계산

monitor.sh: top -bn1 | grep "Cpu(s)" | awk '{print \$2}' → 시스템 전체 CPU 사용률(%) 추출

ps -ef | grep agent-leak-app 의 %CPU: 특정 프로세스 한 개의 점유율(시스템 대비 비율)

agent_app.log의 actual≈99.7%는 os.times()로 '이 프로세스가 실제로 CPU를 얼마나 썼는지' 직접 측정한 값

→ top/ps는 '시스템에서 본 시점의 스냅샷', os.times()는 '프로세스 자신의 누적 사용량 기반 계산' — 측정 주체가 다름

항목 2-③ — "살아있지만 멈춘 상태" 진단 도구 사용 순서

Q. 프로세스가 "살아있지만 멈춰있는 상태"를 진단하기 위해 어떤 도구를 어떤 순서로 사용했는가?

- ① `ps -ef | grep agent-leak-sim` → PID 존재 여부 확인 (프로세스 생존 여부 1차 판단)
- ② 잠시 후 동일 명령 재실행 → TIME 컬럼 비교 → 값이 변화 없으면 'CPU 작업을 안 하고 있다'
- ③ `agent_app.log` 확인 (`tail -f`) → 마지막 로그 이후 추가 출력이 없으면 '응답이 없다'
- ④ (선택) `top -H -p <PID>` → 스레드별 상태(S=Sleep, D=Disk wait 등)를 확인해 어떤 스레드가 대기 중인지 특정

관찰 항목	정상 프로세스	Deadlock(무응답) 상태
PID (<code>ps -ef</code>)	존재	존재 (그대로 살아있음)
TIME 변화 (반복 <code>ps</code>)	계속 증가	변화 없음 (작업 정지)
로그 출력 (<code>agent_app.log</code>)	계속 갱신	특정 시점 이후 완전히 멈춤

결론: ①~③ 세 가지가 모두 '정지' 패턴으로 동시에 나타날 때 → "PID는 살아있지만 멈춘 Deadlock 상태"로 판단

항목 3-①② — 보호 정책이 프로세스를 강제 종료하는 이유

Q1. 메모리 누수 시 MemoryGuard가 해당 프로세스를 강제 종료하는 이유는?

메모리는 OS 전체가 공유하는 자원. 한 프로세스가 한계까지 계속 누적하면 다른 프로세스가 할당받을 메모리가 부족해져 시스템 전체가 느려지거나 멈춤(OOM) → "이 프로세스 1개"를 희생시켜 "시스템 전체"를 지킴
보호장치가 없다면 OS의 OOM Killer가 더 무차별적으로(다른 프로세스까지) 개입할 수 있음

Q2. CPU 과점유 시 단일 프로세스 종료가 시스템 보호에 필요한 이유는?

CPU는 여러 프로세스가 시분할(time-sharing)하는 자원. 1개 프로세스가 코어를 독점하면 다른 모든 프로세스의 응답이 지연됨 → SIGTERM(정상 종료 요청)으로 그 1개를 멈춰 "다수"의 정상 동작을 보장
SIGTERM은 마무리 작업 후 종료를 요청하는 신호 (즉시·강제 종료인 SIGKILL과 구분)

공통 원리: "소수(프로세스 1개)의 희생으로 다수(시스템 전체)의 안정성을 지킨다"

항목 3-③④ — Deadlock 원리(상호배제·순환대기) & 로그 추적 과정

Q3. Deadlock이 발생하는 원리를 "상호 배제"와 "순환 대기"로 설명하면 ?

상호 배제(Mutual Exclusion): threading.Lock()은 한 번에 한 스레드만 보유 가능 → Lock-1, Lock-2 각각 동시에 하나의 스레드만 가질 수 있음

순환 대기(Circular Wait): A는 Lock-2(B 보유)를 기다리고, B는 Lock-1(A 보유)를 기다림 → $A \rightarrow B \rightarrow A$ 순환 구조 → 누구도 자신의 락을 먼저 양보하지 않으면 영원히 풀리지 않음

Q4. 로그에서 스레드 간 순환 의존 관계($A \rightarrow B, B \rightarrow A$)를 어떻게 파악했는가 ?

[Thread-A] Acquired Lock-1, WAITING for Lock-2... ← A: Lock-1 보유, Lock-2 대기
[Thread-B] Acquired Lock-2, WAITING for Lock-1... ← B: Lock-2 보유, Lock-1 대기

- ① 각 줄에서 'Acquired X' = 보유 자원, 'WAITING for Y' = 대기 자원으로 분리해서 읽
- ② A가 기다리는 Lock-2의 현재 소유자(B의 로그에서 'Acquired Lock-2')를 대조 → $A \rightarrow B$
- ③ B가 기다리는 Lock-1의 현재 소유자(A의 로그에서 'Acquired Lock-1')를 대조 → $B \rightarrow A$
- ④ 두 화살표를 합치면 $A \rightarrow B \rightarrow A$ 순환 그래프 완성 → 순환 대기(Circular Wait) = Deadlock 확정

항목 4-① — 운영 환경 사전탐지를 위한 monitor.sh 개선 제안

Q. agent-leak-app이 실제 운영 서버에서 동작 중이라면, OOM을 장애 전에 탐지하기 위해 monitor.sh를 어떻게 개선할까?

현재 monitor.sh	개선 제안
MEM_USAGE(%)를 1분마다 1줄 기록, 임계치(10%) 초과 시 1회 WARNING만 출력	최근 N회(예: 5분)의 MEM_USAGE 추이를 비교 → '지속 증가' 패턴 자체를 탐지 (단발성 초과와 구분)
시스템 전체 메모리 사용률(%)만 측정	대상 프로세스의 RSS를 함께 기록 (ps -o rss= -p \$PID) → '어떤 프로세스'가 누수 원인인지 특정 가능
임계치 초과 시 로그 파일에만 WARNING 기록	증가율 기반 예측: "이 속도면 N분 후 한계 도달" 사전 경고 + Slack/메일 등 알림 채널 연동

항목 4-② — 가장 치명적인 장애: Deadlock

Q. OOM / CPU Spike / Deadlock 중 실제 서비스 환경에서 가장 치명적인 것은? 이유와 예방법은?

OOM / CPU Spike: 프로세스가 "스스로 종료"됨 → 헬스체크(PID/포트)가 즉시 실패로 감지 → supervisor/systemd가 자동 재시작 → 장애가 짧고 명확하게 드러남

Deadlock: 프로세스는 "살아있음"(PID 존재, 포트도 열린 채 유지될 수 있음) → 단순 PID/포트 체크는 정상으로 오인 → 감지 자체가 늦어지고, 그동안 들어오는 모든 요청이 무한 대기에 쌓임

"보이지 않는 장애"가 가장 위험하다 — 감지가 늦을수록 영향(대기 요청 수)이 누적된다

예방법 ① 코드 레벨: 모든 스레드가 락을 동일한 순서(Lock-1 → Lock-2)로만 획득 + lock.acquire(timeout=...) 적용

예방법 ② 운영 레벨: 헬스체크에 "응답 시간(latency)"을 포함 — 단순 PID/포트가 아닌 실제 요청-응답 사이클로 검증

항목 4-③ — OOM + Deadlock 동시 발생 시 트러블슈팅 순서

Q. 동일 서버에서 OOM과 Deadlock이 동시에 발생했다면 어떤 순서로 처리하며, 그 근거는?

순서	조치	이유
1	Deadlock 프로세스의 상태 스냅샷 확보 (ps -ef, top -H, 스레드 덤프)	재시작 전에 원인 분석용 증거를 먼저 확보
2	Deadlock 프로세스 재시작 → 서비스 응답 최우선 복구	무응답은 서비스 영향이 즉시·전면적이며 자가 복구되지 않음 (최우선)
3	OOM 로그/메모리 추세를 별도로 분석 (누수 원인 추적)	OOM은 MemoryGuard의 자가 종료로 1차 방어가 이미 동작 중 → 상대적으로 여유 있음

우선순위 판단 기준 = 서비스 영향도 × 자가복구 가능성 → Deadlock(영향 큼·자가복구 불가) 우선

항목 4-④ — 장애 유형별 코드 레벨 개선안

Q. 환경변수 조정은 임시 조치였다. 소스 코드를 직접 수정한다면 각 장애 유형별로 어떤 개선을 하겠는가?

장애 유형	코드 레벨 개선안
OOM (Memory Leak)	사용 후 <code>del</code> 로 명시적 해제 / 캐시에 TTL·최대 크기 설정 / <code>with</code> 컨텍스트 매니저로 리소스 자동 정리
CPU Spike	<code>busy-loop</code> 을 작은 단위로 분할 + 중간에 <code>time.sleep(0)/yield</code> 삽입 / 무거운 연산은 별도 워커 프로세스로 분산
Deadlock	모든 스레드가 락을 'Lock-1 → Lock-2' 고정 순서로만 획득 / <code>lock.acquire(timeout=...)</code> + 재시도 / 가능하면 단일 락으로 통합

항목 4-⑤ — 다시 수행한다면 다르게 접근할 점

Q. 이 미션을 처음부터 다시 수행한다면, 트러블슈팅 과정에서 어떤 점을 다르게 접근하겠는가?

- ① agent_app.log(앱 로그)뿐 아니라 monitor.log(시스템 리소스 시계열)도 함께 수집하여, 앱 로그와 시스템 지표를 같은 시간대로 대조
- ② 케이스별 환경변수를 더 작게 설정(예: MEMORY_LIMIT을 더 낮춤)하여 재현 대기 시간을 단축
- ③ ps/top 등 증거 캡처(스크린샷)를 각 단계 직후 즉시 저장 → 보고서 작성 시 누락 방지
- ④ Before/After 비교 시, 조치 후에도 결국 재발하는지까지 한 번 더 관찰 → "임시방편 vs 근본 해결"을 더 명확히 구분

정리 — 평가문항 대비 핵심 메시지

- 항목 1: 3가지 장애(OOM/CPU/Deadlock) 재현 + Before/After 증거 → GitHub Issue 3건으로 모두 충족
- 항목 2: 사용한 명령어(`free`, `top -bn1`, `ps -eo`, `os.times()`)의 옵션·측정 대상까지 구체적으로 설명 가능
- 항목 3: 보호 정책이 "왜" 강제 종료하는지, Deadlock의 "상호배제 + 순환대기" 원리와 로그 추적 과정을 설명 가능
- 항목 4: 운영 환경 적용 시 모니터링 개선 / 우선순위 판단 / 코드 레벨 개선까지 구체적 제안 가능

"장애 = 감으로 추측하는 것" → "장애 = 데이터로 증명하고, 원리를 설명하고, 개선까지 제안하는 것"